JAVA · INTELLIJ · JUNIT 5

Wild West Shootout Simulator

A line-by-line code walkthrough and program structure reference for the circular cowboy-shootout simulation: rules engine, JSON protocol writer, SHA-256 checksum, console narration, fairness statistics tool, and its JUnit test suite.

> **Package:** cowboyshootout
>
> **Language:** Java 17+
>
> **Build:** plain javac (program) / Maven (tests)
>
> **Files covered:** 9 source files

1. Program Structure

The program simulates a circular gunfight: cowboys start at equal hit points, the active shooter's own current hp (odd/even) decides whether he fires at his left or right neighbour, kills close the circle and let the same shooter fire again, and the fight ends when one cowboy is left. Every shot is logged, written out as a JSON protocol file, and the file is hashed with SHA-256 so the record can't be silently altered afterwards.

1.1 Directory layout

```
cowboy-shootout-simulator/
├── pom.xml                       # Maven config, only needed to run the tests
├── README.md                     # usage, design notes, fairness write-up
├── sample-output/                # one pre-generated example run (seed 42)
│   ├── protocol_8cowboys_seed42.json
│   └── protocol_8cowboys_seed42.json.sha256
└── src/
    ├── main/java/cowboyshootout/
    │   ├── Cowboy.java             # one fighter: id, name, hp, prev/next link
    │   ├── Game.java               # the rules; nests Side, Shot, Result
    │   ├── Protocol.java           # JSON writer + SHA-256 checksum
    │   ├── Main.java               # CLI entry point
    │   └── Stats.java              # optional: fairness statistics tool
    └── test/java/cowboyshootout/
        ├── CowboyTest.java
        ├── GameTest.java
        └── ProtocolTest.java
```

1.2 What each file is responsible for

File	Responsibility
`Cowboy.java`	Data plus one rule: a fighter's id, display name, current hp, and links to its two circle neighbours (`prev`/`next`). `hpIsEven()` is the single piece of logic living here — it implements the task's "even hp = shoot right" rule.
`Game.java`	The actual simulation. `play(seed, logShots)` builds the circle, runs the shoot-out loop until one cowboy remains, and returns a `Result`. Also declares three small nested types it works with: `Side` (LEFT/RIGHT), `Shot` (one protocol line) and `Result` (the whole game's outcome).
`Protocol.java`	Turns a `Game.Result` into a UTF-8 JSON file (hand-written, no external JSON library) and computes the file's SHA-256 checksum using the JDK's own `MessageDigest`.
`Main.java`	The command-line entry point: reads the cowboy count (and optional seed), plays one game, prints it as a narrated story, calls `Protocol`, and prints the resulting file path and

	checksum.
`Stats.java`	Not required by the task — a Monte-Carlo tool that plays many games fast (protocol logging switched off) and reports win rate by seat position, used to answer "is the game fair?" with real numbers.
`pom.xml`	A minimal Maven file whose only job is to pull in JUnit 5 (test scope) so `mvn test` works and IntelliJ can auto-detect the tests. The program itself has zero dependencies and still compiles with plain `javac`.
`CowboyTest.java`	Unit tests for `Cowboy`: name formatting and hp-parity.
`GameTest.java`	Unit tests for the actual game rules: reproducibility, damage bounds, kill counts, the "first shot always goes right" fact, and constructor validation.
`ProtocolTest.java`	Unit tests for the JSON writer and checksum: file shape, and checksum stability/uniqueness.

1.3 How a run flows

java cowboyshootout.Main 8 42

Main.main() → **new Game(8, 10)** → **game.play(42, true)** runs the whole fight inside Game and returns a **Game.Result** (built from Cowboy objects linked in a circle, producing a list of Game.Shot records) → Main prints that Result as a story (tellStory) → Main hands the same Result to **Protocol.write()** to create the JSON file → **Protocol.sha256()** hashes that file → Main prints the winner, file path and checksum.

Stats follows the same Main → Game.play() path, just in a loop of thousands of games with logging off, tallying who won relative to seat position instead of printing a story.

The **tests** call `Game`, `Cowboy` and `Protocol` directly (no `Main` involved) so they can assert on the data before it becomes text.

> **Why a circular doubly-linked list?** Each `Cowboy` holds `prev`/`next` pointers to its neighbours. Closing the circle after a kill is then a two-line pointer swap (`target.prev.next = target.next; target.next.prev = target.prev;`) instead of shifting an array or list — it mirrors the physical picture of two neighbours stepping toward each other exactly.

2. Line-by-Line Code Walkthrough

Each file below shows its full source with line numbers, followed by an annotation list. Every tag on the left names the exact lines it explains.

2.1 Cowboy.java

One fighter standing in the circle

```java
package cowboyshootout;

// One fighter standing in the circle. The circle itself is just these
// objects linked to each other (prev/next), so removing a dead cowboy
// is as simple as pointing his neighbors at each other.
class Cowboy {

    int id;
    String name;
    int hp;
    Cowboy prev;
    Cowboy next;

    Cowboy(int id, int hp) {
        this.id = id;
        this.name = "Cowboy-" + id;
        this.hp = hp;
    }

    boolean hpIsEven() {
        return hp % 2 == 0;
    }

    @Override
    public String toString() {
        return name;
    }
}
```

L1 Package declaration — every file in the project shares `cowboyshootout`, which is why the classes can refer to each other by plain name without importing one another.

L3-5 A comment explaining the key design idea: there is no separate array/list representing the circle. The circle *is* these `Cowboy` objects linked to each other.

L6 Declares the class. No `public` keyword — it only needs to be visible to other classes in the same package, which is all that ever uses it.

L8-12 Five fields living on every `Cowboy` object: seat number (`id`), display name, current health, and two pointers (`prev` / `next`) to the neighbours on either side. These pointers are what make the circle real.

L14-18 The constructor, run by `new Cowboy(id, hp)`. It stores the id and hp, and builds the display name from the id (id 3 becomes `"Cowboy-3"`). `this.id = id` distinguishes the field from the constructor parameter of the same name.

L20-22 `hpIsEven()` is the one piece of logic in this class. `%` is the remainder operator; `hp % 2 == 0` is true exactly when `hp` is even. This directly implements the task's rule: if this number is even he shoots right.

L24-27 Overrides `toString()`, the method Java calls automatically whenever an object is printed or glued into a string with `+`. Without this override, printing a `Cowboy` would show something like `cowboyshootout.Cowboy@1b6d3586`; with it, it shows `"Cowboy-3"`. `@Override` makes the compiler check that this really does override an existing method.

2.2 Game.java

The rules engine — plays one game start to finish

```java
package cowboyshootout;

import java.util.ArrayList;
import java.util.List;
import java.util.Random;

// runs one shootout from start to finish
class Game {

    static final int MIN_DMG = 1;
    static final int MAX_DMG = 5;

    int count;
    int startHp;

    Game(int count, int startHp) {
        if (count < 1) throw new IllegalArgumentException("need at least 1 cowboy");
        if (startHp < 1) throw new IllegalArgumentException("need at least 1 hp");
        this.count = count;
        this.startHp = startHp;
    }

    // logShots=false skips building the Shot list, used when we just want
    // to know who won (e.g. running thousands of games for Stats.java)
    Result play(long seed, boolean logShots) {
        Random rng = new Random(seed);
```

```java

        // seat the cowboys in a circle
        Cowboy[] seats = new Cowboy[count];
        for (int i = 0; i < count; i++) {
            seats[i] = new Cowboy(i + 1, startHp);
        }
        for (int i = 0; i < count; i++) {
            seats[i].next = seats[(i + 1) % count];
            seats[i].prev = seats[(i - 1 + count) % count];
        }

        List<String> order = new ArrayList<>();
        for (Cowboy c : seats) order.add(c.name);

        Cowboy active = seats[rng.nextInt(count)];
        String starter = active.name;

        List<Shot> shots = logShots ? new ArrayList<>() : List.of();
        int alive = count;
        int shotNum = 0;

        while (alive > 1) {
            Side side = active.hpIsEven() ? Side.RIGHT : Side.LEFT;
            Cowboy target = (side == Side.RIGHT) ? active.next : active.prev;

            int dmg = MIN_DMG + rng.nextInt(MAX_DMG - MIN_DMG + 1);
            int before = target.hp;
            target.hp -= dmg;
            boolean killed = target.hp <= 0;
            shotNum++;

            if (logShots) {
                shots.add(new Shot(shotNum, active.name, active.hp, side, target.name,
                        dmg, before, Math.max(0, target.hp), killed));
            }

            if (killed) {
                target.prev.next = target.next;
                target.next.prev = target.prev;
                alive--;
                // active shoots again, so we just loop without switching active
            } else {
                active = target;
            }
        }

        Result r = new Result();
        r.count = count;
        r.startHp = startHp;
        r.seed = seed;
        r.order = order;
```

```
        r.starter = starter;
        r.shots = shots;
        r.winner = active.name;
        r.winnerHp = active.hp;
        return r;
    }

    // which neighbor got shot at
    enum Side {
        LEFT,
        RIGHT
    }

    // one line of the protocol: who shot who, and what happened
    static class Shot {
        int num;
        String shooter;
        int shooterHp;
        Side side;
        String target;
        int dmg;
        int hpBefore;
        int hpAfter;
        boolean killed;

        Shot(int num, String shooter, int shooterHp, Side side, String target,
             int dmg, int hpBefore, int hpAfter, boolean killed) {
            this.num = num;
            this.shooter = shooter;
            this.shooterHp = shooterHp;
            this.side = side;
            this.target = target;
            this.dmg = dmg;
            this.hpBefore = hpBefore;
            this.hpAfter = hpAfter;
            this.killed = killed;
        }
    }

    // everything that happened during one game
    static class Result {
        int count;
        int startHp;
        long seed;
        List<String> order;
        String starter;
        List<Shot> shots;
        String winner;
        int winnerHp;
    }
}
```

L1-5	Package, then three JDK imports: `ArrayList` / `List` for ordinary growable lists, and `Random` for the seeded random number generator.
L8	The class holding the whole simulation.
L10-11	`static final` constants shared by every `Game` instance — the damage bounds from the task (loses 1-5 healthpoints).
L13-14	Fields: how many cowboys, and their starting hp.
L16-21	Constructor with a sanity check. `new Game(0, 10)` or `new Game(5, -1)` throws `IllegalArgumentException` immediately with a clear message, instead of silently misbehaving later.
L25	`play(seed, logShots)` runs one entire game and hands back a `Result`. `logShots=false` skips building the detailed shot list — used by `Stats.java`, which plays thousands of games and only cares who won.
L26	`new Random(seed)`: giving it a fixed seed makes the random sequence exactly reproducible — the same seed always plays out identically.
L29-32	Allocates an array of `count` empty slots and fills each with a new `Cowboy`. `i+1` turns index 0,1,2,... into ids 1,2,3,... (`Cowboy-1`, `Cowboy-2`, ...).
L33-36	Wires up the circle. For cowboy `i`, `next` is cowboy `i+1` — except at the last seat, where `% count` wraps the index back to 0, closing the ring. `(i - 1 + count) % count` does the same going backward (the `+ count` avoids a negative number when `i` is 0).
L38-39	Builds a plain list of names in seating order, purely for the record (the initial layout gets printed and saved).
L41-42	`rng.nextInt(count)` picks a uniformly random index — the random-starting-cowboy rule — and `active` becomes the running whoever's-turn-it-is variable.
L44-46	`shots` is the growable log (or an unused empty placeholder if not logging). `alive` tracks the living headcount; `shotNum` numbers shots 1,2,3,...
L48	The whole game is this one loop: keep firing while more than one cowboy is alive — the task's exact ending condition.
L49-50	The core rule, in two lines: even current hp gives `Side.RIGHT`, odd gives `Side.LEFT`; then `target` is whichever neighbour that side points to.
L52	Random damage: `MAX_DMG - MIN_DMG + 1` is 5, so `rng.nextInt(5)` gives 0-4, and adding `MIN_DMG` shifts it to 1-5 inclusive.
L53-56	Remembers hp before the hit, applies the damage, checks whether it was fatal (`hp <= 0`, exactly the task's kill condition), and advances the shot counter.

L58-61 If logging, builds one `Shot` record with everything that just happened. `Math.max(0, target.hp)` reports 0 instead of a negative number for a fatally overkilled target — real health doesn't go below zero.

L63-70 Closing the circle. If the target died: point its left neighbour's `next` straight at its right neighbour, and vice versa — the dead cowboy is now unreachable from either side. Crucially, `active` is **not** reassigned here, so the same shooter fires again next iteration, exactly as the task specifies. If the target survived instead, `active = target` passes the turn on.

L73-82 Once the loop ends, exactly one cowboy is left — and because `active` was never moved away from a survivor, `active` *is* that last cowboy. Everything is packaged into a `Result` and handed back to the caller.

L85-89 `enum Side`: a fixed two-value type (`LEFT` / `RIGHT`) — safer than a raw `String` or `boolean`, since the compiler catches typos.

L91-115 `Shot`: a small data bundle for one protocol line (nine fields), built in one call via its constructor, e.g. `new Shot(shotNum, active.name, ...)` on L59.

L117-127 `Result`: a data bundle for the whole game's outcome, filled in gradually (L73-82) rather than via a constructor. Both `Shot` and `Result` are `static` nested classes — that keyword means they don't secretly hold a reference back to a particular `Game` instance, so other files use them as ordinary types named `Game.Shot` / `Game.Result`.

2.3 Protocol.java

Writes the JSON file and hashes it

```java
package cowboyshootout;

import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.security.MessageDigest;
import java.util.HexFormat;
import java.util.List;

// writes the game to a JSON file and hashes it, so the sheriff has proof
// nobody edited the story afterward
class Protocol {

    static Path write(Game.Result r, Path file) throws IOException {
        StringBuilder out = new StringBuilder();
        out.append("{\n");
        out.append("  \"cowboys\": ").append(r.count).append(",\n");
        out.append("  \"startHp\": ").append(r.startHp).append(",\n");
```

```
        out.append("  \"seed\": ").append(r.seed).append(",\n");
        out.append("  \"order\": ").append(toJsonArray(r.order)).append(",\n");
        out.append("  \"starter\": ").append(quote(r.starter)).append(",\n");

        out.append("  \"shots\": [\n");
        for (int i = 0; i < r.shots.size(); i++) {
            Game.Shot s = r.shots.get(i);
            out.append("    {");
            out.append("\"num\":").append(s.num).append(",");
            out.append("\"shooter\":").append(quote(s.shooter)).append(",");
            out.append("\"shooterHp\":").append(s.shooterHp).append(",");
            out.append("\"side\":").append(quote(s.side.name())).append(",");
            out.append("\"target\":").append(quote(s.target)).append(",");
            out.append("\"dmg\":").append(s.dmg).append(",");
            out.append("\"hpBefore\":").append(s.hpBefore).append(",");
            out.append("\"hpAfter\":").append(s.hpAfter).append(",");
            out.append("\"killed\":").append(s.killed);
            out.append("}");
            out.append(i < r.shots.size() - 1 ? ",\n" : "\n");
        }
        out.append("  ],\n");

        out.append("  \"winner\": ").append(quote(r.winner)).append(",\n");
        out.append("  \"winnerHp\": ").append(r.winnerHp).append("\n");
        out.append("}\n");

        Files.writeString(file, out, StandardCharsets.UTF_8);
        return file;
    }

    static String sha256(Path file) throws IOException {
        try {
            MessageDigest digest = MessageDigest.getInstance("SHA-256");
            byte[] hash = digest.digest(Files.readAllBytes(file));
            return HexFormat.of().formatHex(hash);
        } catch (java.security.NoSuchAlgorithmException e) {
            throw new AssertionError(e); // SHA-256 always exists on the JVM
        }
    }

    private static String toJsonArray(List<String> values) {
        StringBuilder sb = new StringBuilder("[");
        for (int i = 0; i < values.size(); i++) {
            sb.append(quote(values.get(i)));
            if (i < values.size() - 1) sb.append(", ");
        }
        return sb.append("]").toString();
    }

    private static String quote(String s) {
        StringBuilder sb = new StringBuilder("\"");
```

```
71          for (char c : s.toCharArray()) {
72              switch (c) {
73                  case '"' -> sb.append("\\\"");
74                  case '\\' -> sb.append("\\\\");
75                  default -> sb.append(c);
76              }
77          }
78          return sb.append("\"").toString();
79      }
80  }
```

L1-9 Package, then JDK imports: file I/O (`Files`, `Path`, `StandardCharsets`), cryptography (`MessageDigest`), hex formatting (`HexFormat`), and `List`. Nothing external — the whole file uses only the standard library.

L15 `write(...)` is `static` — called as `Protocol.write(...)`, no `Protocol` object needed. `throws IOException` means a disk problem is not handled here; it's passed up to the caller (`Main`), which lets the program simply crash with a clear trace if something goes wrong — fine for a small CLI tool.

L16 `StringBuilder` is the efficient way to build a long piece of text incrementally (plain `String +=` in a loop would copy the whole string every time).

L17-22 Hand-writes the opening JSON object and its scalar fields. Numbers go in directly; strings are wrapped with the `quote(...)` helper (L69-79), which also escapes any quote or backslash inside them.

L24-39 Opens the `"shots"` array, then loops over every recorded shot and writes it as one compact object per line (no internal newlines — a deliberately terse style). The line adds a comma after every shot except the last, since JSON forbids a trailing comma before the closing bracket.

L42-44 Writes the winner's name and remaining hp, then closes the outer object.

L46-47 `Files.writeString(file, out, StandardCharsets.UTF_8)` is a single JDK call that opens, writes and closes the file as UTF-8 text. The `Path` is returned so `Main` has it on hand.

L50-58 `sha256(...)`: `MessageDigest.getInstance("SHA-256")` asks the JVM for a built-in SHA-256 hasher (no external library). `Files.readAllBytes(file)` re-reads the file from disk, `digest(...)` hashes those raw bytes, and `HexFormat.of().formatHex(hash)` renders them as the familiar hex string. The catch block can never actually run — SHA-256 is guaranteed to exist on every standard JVM — so it just satisfies the compiler by re-throwing as an `AssertionError`.

L60-67 `toJsonArray(...)`: a small private helper turning a list like ["Cowboy-1", "Cowboy-2"] into the literal JSON text for that array.

L69-79 `quote(...)`: wraps a string in quotes, escaping the two characters that would otherwise break JSON — a literal quote becomes an escaped quote, a literal backslash becomes an escaped backslash. The arrow `switch (c) -> ...` is Java's modern switch-statement syntax.

2.4 Main.java

CLI entry point — ties everything together

```java
package cowboyshootout;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.security.SecureRandom;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

// Program arguments: <numberOfCowboys> [seed]
// Example: 8        -> 8 cowboys, random seed
//          8 42      -> 8 cowboys, fixed seed (reproducible run)
public class Main {

    static final int START_HP = 10;

    public static void main(String[] args) throws IOException {
        if (args.length < 1) {
            System.out.println("usage: java cowboyshootout.Main <numberOfCowboys> [seed]");
            return;
        }

        int cowboys = Integer.parseInt(args[0].trim());
        long seed = args.length >= 2 ? Long.parseLong(args[1].trim()) : new SecureRandom().nextLong();

        System.out.println("Wild West Shootout - " + cowboys + " cowboys, " + START_HP + " hp each, seed " + seed);
        System.out.println();

        Game game = new Game(cowboys, START_HP);
        Game.Result result = game.play(seed, true);

        tellStory(result);

        String stamp = LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyyMMdd_HHmmss"));
        Path file = Path.of("protocol_" + cowboys + "cowboys_" + stamp + ".json");
        Protocol.write(result, file);
        String hash = Protocol.sha256(file);
```

```
        Files.writeString(Path.of(file + ".sha256"), hash + "  " + file.getFileName() + "\n");

        System.out.println();
        System.out.println(result.winner + " wins with " + result.winnerHp + " hp left! (" + result.shots.size() + " shots fired)");
        System.out.println();
        System.out.println("protocol file : " + file.toAbsolutePath());
        System.out.println("sha-256       : " + hash);
    }

    // print the fight as a little story instead of a bare table of numbers
    static void tellStory(Game.Result result) {
        System.out.println("The circle: " + String.join(" - ", result.order));
        System.out.println(result.starter + " draws first.");
        System.out.println();

        for (Game.Shot s : result.shots) {
            String turn = s.side == Game.Side.RIGHT ? "turns right" : "turns left";
            String line = s.num + ". " + s.shooter + " (" + s.shooterHp + " hp) " + turn
                    + " and fires at " + s.target + " -> " + s.dmg + " dmg, "
                    + s.target + " drops to " + s.hpAfter + " hp";
            if (s.killed) {
                line += ". " + s.target + " falls! The circle closes and " + s.shooter + " shoots again.";
            }
            System.out.println(line);
        }
    }
}
```

L13 This class *is* `public` — it must be, since it's the entry point run from outside the package (`java cowboyshootout.Main`).

L17 `main(String[] args)` is the fixed signature the JVM looks for. `args` holds whatever text was typed after the class name on the command line.

L18-21 No arguments given: print a usage hint and `return` (exit the program) instead of crashing further down.

L23-24 Command-line arguments always arrive as text, so `Integer.parseInt` / `Long.parseLong` convert them. If a seed was given, use it; otherwise generate one from `SecureRandom` — used only to pick this one starting number, after which `Game`'s own seeded `Random` takes over.

L26-27 Prints the run's header line to the console.

L29-30 Creates the `Game` and plays it with `logShots=true` — the full story is wanted for both the console and the file.

L32 Hands the result to `tellStory` (defined below) to narrate the fight.

L34-38	Builds a timestamped filename so repeated runs never overwrite each other, writes the protocol, hashes it, and writes a `.sha256` sidecar file in the standard `sha256sum -c` format.
L40-44	Final console summary: winner, shot count, file path, checksum.
L48-51	`tellStory` starts by printing the seating order (`String.join` glues the names together with a separator between each) and who drew first.
L53-62	Loops through every recorded shot and builds one narrative sentence per shot with plain string concatenation (`+=` appends onto the end of `line`), adding an extra sentence when that shot was a kill.

2.5 Stats.java

Optional: Monte-Carlo fairness check

```java
package cowboyshootout;

import java.security.SecureRandom;

// not needed for the actual assignment, just here to check whether the
// game is fair: runs a lot of games and counts wins by seat compared to
// the cowboy who started.
// Program arguments: <numberOfCowboys> [numberOfGames]
public class Stats {

    static final int START_HP = 10;

    public static void main(String[] args) {
        int cowboys = args.length >= 1 ? Integer.parseInt(args[0].trim()) : 8;
        int games = args.length >= 2 ? Integer.parseInt(args[1].trim()) : 200_000;

        Game game = new Game(cowboys, START_HP);
        SecureRandom seedGen = new SecureRandom();

        long[] winsBySeat = new long[cowboys]; // 0 = the starter, 1 = starter's right neighbor, ...
        int sample = Math.min(games, 20_000);
        long shotsInSample = 0;
        long firstShotRight = 0;

        for (int i = 0; i < games; i++) {
            boolean log = i < sample;
            Game.Result r = game.play(seedGen.nextLong(), log);

            int winnerSeat = Integer.parseInt(r.winner.substring("Cowboy-".length()));
            int starterSeat = Integer.parseInt(r.starter.substring("Cowboy-".length()));
            winsBySeat[Math.floorMod(winnerSeat - starterSeat, cowboys)]++;
```

```
32
33              if (log) {
34                  shotsInSample += r.shots.size();
35                  if (!r.shots.isEmpty() && r.shots.get(0).side == Game.Side.RIGHT)
firstShotRight++;
36              }
37          }
38
39          System.out.println(cowboys + " cowboys, " + games + " games (" + sample + " sampled
for extra stats)");
40          System.out.println();
41          System.out.printf("first shot went RIGHT: %d / %d (%.1f%%)%n", firstShotRight,
sample, 100.0 * firstShotRight / sample);
42          System.out.printf("average shots per game: %.1f%n", (double) shotsInSample /
sample);
43          System.out.println();
44          System.out.println("win rate by seat, counted from the starter (seat 0 = starter,
seat 1 = starter's right neighbor):");
45          double fairShare = 100.0 / cowboys;
46          for (int seat = 0; seat < cowboys; seat++) {
47              double pct = 100.0 * winsBySeat[seat] / games;
48              System.out.printf("  seat %-2d: %5.1f%%   (fair share: %.1f%%)%n", seat, pct,
fairShare);
49          }
50      }
51  }
```

L5-8 This whole file is extra, not required by the task — the comment says so up front. It exists purely to back the is-the-game-fair discussion with real numbers instead of just intuition.

L13-15 Reads two optional arguments: how many cowboys per game (default 8) and how many games to simulate (default 200,000), using a ternary to supply the default when the argument was omitted.

L17-18 One reusable `Game` (same cowboy count/hp every trial) and a `SecureRandom` used just to generate a fresh seed for each game — so every one of the thousands of games is genuinely independent.

L20-23 `winsBySeat[k]` will count how often the cowboy k seats clockwise from that game's starter wins. Only the first 20,000 games (`sample`) get the expensive full shot log, to keep the whole run fast.

L25-37 The main loop: play one game (logging only for the sampled subset), then figure out the winner's seat offset from the starter using `Math.floorMod` (handles the wrap-around correctly even when the subtraction goes negative) and tally it. For sampled games, also count total shots and whether the very first shot went `RIGHT` .

L39-49 Prints a summary: how often the first shot was RIGHT (should be 100% — every cowboy starts at an even 10 hp), average shots per game, and a per-seat table comparing the actual win rate against the fair share every seat would get if position didn't matter.

2.6 pom.xml

Just enough Maven to run the tests

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0">
    <modelVersion>4.0.0</modelVersion>

    <groupId>cowboyshootout</groupId>
    <artifactId>cowboy-shootout-simulator</artifactId>
    <version>1.0</version>
    <packaging>jar</packaging>

    <properties>
        <maven.compiler.release>17</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <junit.version>5.10.2</junit.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>${junit.version}</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <finalName>cowboy-shootout-simulator</finalName>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-plugin</artifactId>
                <version>3.2.5</version>
            </plugin>
        </plugins>
    </build>
</project>
```

L1-2 The root `<project>` element and Maven's fixed `modelVersion` — boilerplate every pom.xml starts with.

L4-7 The project's coordinates: a group id, artifact id, version, and jar packaging (the default for a plain Java library/program — no web server, no special archive type).

L9-13 `maven.compiler.release 17` targets a widely-available LTS Java version (the code doesn't use anything newer). `sourceEncoding UTF-8` tells Maven how to read the .java files themselves. `junit.version` is just a named constant reused below.

L15-22 The only dependency: `junit-jupiter` (JUnit 5), scoped to `test` — meaning it's on the classpath only while compiling/running tests, never bundled with the actual program. This is what keeps Cowboy/Game/etc. dependency-free.

L24-33 Pins an explicit, modern version of the `maven-surefire-plugin` (the plugin that actually runs tests during `mvn test`) so JUnit 5 tests are discovered and executed reliably regardless of which Maven version is installed.

2.7 CowboyTest.java

Unit tests for Cowboy

```java
package cowboyshootout;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertFalse;
import static org.junit.jupiter.api.Assertions.assertTrue;

class CowboyTest {

    @Test
    void nameIsBuiltFromId() {
        Cowboy c = new Cowboy(3, 10);
        assertEquals("Cowboy-3", c.name);
    }

    @Test
    void toStringIsJustTheName() {
        Cowboy c = new Cowboy(5, 10);
        assertEquals("Cowboy-5", c.toString());
    }

    @Test
    void hpIsEvenTrueForEvenHp() {
        Cowboy c = new Cowboy(1, 10);
        assertTrue(c.hpIsEven());
    }

    @Test
    void hpIsEvenFalseForOddHp() {
        Cowboy c = new Cowboy(1, 9);
        assertFalse(c.hpIsEven());
    }
}
```

L3-7	Imports the JUnit 5 `@Test` annotation and three assertion methods as static imports, so they can be called bare (`assertEquals(...)`) instead of `Assertions.assertEquals(...)`.
L11-15	`@Test` marks a method as a runnable test case. This one checks the constructor builds the name as `"Cowboy-" + id`.
L17-21	Confirms `toString()` returns exactly the name, nothing else.
L23-27	Confirms `hpIsEven()` is true for an even hp (10).
L29-33	Confirms `hpIsEven()` is false for an odd hp (9) — the other half of the same rule.

2.8 GameTest.java

Unit tests for the game rules

```java
package cowboyshootout;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;

import java.util.ArrayList;
import java.util.List;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;
import static org.junit.jupiter.api.Assertions.assertTrue;

class GameTest {

    @Test
    void oneCowboyWinsWithoutAShotBeingFired() {
        Game.Result r = new Game(1, 10).play(1, true);

        assertEquals("Cowboy-1", r.winner);
        assertEquals(10, r.winnerHp);
        assertTrue(r.shots.isEmpty());
    }

    @Test
    void sameSeedAlwaysPlaysOutTheSameWay() {
        Game game = new Game(8, 10);

        Game.Result first = game.play(42, true);
        Game.Result second = game.play(42, true);

        assertEquals(first.winner, second.winner);
```

```
        assertEquals(first.winnerHp, second.winnerHp);
        assertEquals(shotsAsText(first.shots), shotsAsText(second.shots));
    }

    @ParameterizedTest
    @ValueSource(longs = {1, 2, 3, 42, 100, 999})
    void firstShotAlwaysGoesRightBecauseEveryoneStartsAtAnEvenHp(long seed) {
        Game.Result r = new Game(6, 10).play(seed, true);

        assertEquals(Game.Side.RIGHT, r.shots.get(0).side);
    }

    @Test
    void damageIsAlwaysBetweenOneAndFive() {
        Game.Result r = new Game(10, 10).play(7, true);

        for (Game.Shot s : r.shots) {
            assertTrue(s.dmg >= Game.MIN_DMG && s.dmg <= Game.MAX_DMG,
                    "damage " + s.dmg + " out of range in shot " + s.num);
        }
    }

    @Test
    void hpAfterAShotIsNeverReportedNegative() {
        Game.Result r = new Game(10, 10).play(7, true);

        for (Game.Shot s : r.shots) {
            assertTrue(s.hpAfter >= 0, "negative hp reported in shot " + s.num);
        }
    }

    @Test
    void exactlyOneCowboyIsEliminatedPerSurvivorLessThanTheStart() {
        int count = 8;
        Game.Result r = new Game(count, 10).play(7, true);

        long kills = r.shots.stream().filter(s -> s.killed).count();
        assertEquals(count - 1, kills);
    }

    @Test
    void winnerAlwaysEndsWithPositiveHp() {
        Game.Result r = new Game(8, 10).play(7, true);
        assertTrue(r.winnerHp > 0);
    }

    @Test
    void constructorRejectsZeroCowboys() {
        assertThrows(IllegalArgumentException.class, () -> new Game(0, 10));
    }

```

```
84      @Test
85      void constructorRejectsZeroStartingHp() {
86          assertThrows(IllegalArgumentException.class, () -> new Game(5, 0));
87      }
88
89      private static List<String> shotsAsText(List<Game.Shot> shots) {
90          List<String> lines = new ArrayList<>();
91          for (Game.Shot s : shots) {
92              lines.add(s.num + " " + s.shooter + " " + s.side + " " + s.target + " " + s.dmg
    + " " + s.killed);
93          }
94          return lines;
95      }
96  }
```

L3-5 Imports `@Test`, plus `@ParameterizedTest` / `@ValueSource` which let one test method run several times with different input values.

L7-12 `ArrayList` / `List` for a small helper at the bottom of the file, and three assertion static imports.

L16-23 A 1-cowboy game must end instantly with no shots fired and the sole cowboy still at full hp — the trivial edge case of the game loop never even starting.

L25-35 Plays the same seed twice and checks the two games are identical — not just the winner, but every shot (via the `shotsAsText` helper at L89-95, since `Shot` has no built-in equality check). This proves the RNG seeding actually makes runs reproducible.

L37-43 `@ParameterizedTest` with `@ValueSource` runs this same test body six times, once per seed. Each time, it asserts the very first shot's side is RIGHT — true because every cowboy always starts at hp 10, which is even, so the direction rule is deterministic before any damage has been dealt.

L45-53 Plays a 10-cowboy game and checks every single recorded shot's damage falls within the 1-to-5 range.

L55-62 Checks no shot ever reports a negative `hpAfter` — verifying the clamp in Game.java.

L64-71 Since a game always ends with exactly one survivor, exactly count-1 cowboys must have been killed. This counts how many recorded shots were fatal and compares.

L73-77 The winner must always have positive remaining hp — if they didn't, they'd have been eliminated instead of winning.

L79-87 Two tests confirming `Game`'s constructor validation actually throws `IllegalArgumentException` for 0 cowboys and for 0 starting hp.

L89-95 A private helper turning a list of `Shot` objects into a list of plain strings (one line of key fields per shot) so two shot lists can be compared with ordinary `assertEquals`, since `Game.Shot` doesn't override `equals()`.

2.9 ProtocolTest.java

Unit tests for the JSON writer and checksum

```java
package cowboyshootout;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.io.TempDir;

import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertTrue;

class ProtocolTest {

    @TempDir
    Path tempDir;

    @Test
    void writtenFileIsUtf8AndEndsWithTheExpectedFields() throws IOException {
        Game.Result r = new Game(3, 10).play(5, true);
        Path file = tempDir.resolve("protocol.json");

        Protocol.write(r, file);
        String text = Files.readString(file, StandardCharsets.UTF_8);

        assertTrue(text.startsWith("{"));
        assertTrue(text.trim().endsWith("}"));
        assertTrue(text.contains("\"cowboys\": 3"));
        assertTrue(text.contains("\"winner\": \"" + r.winner + "\""));
        assertBalancedBracesAndBrackets(text);
    }

    @Test
    void sameContentAlwaysHashesToTheSameChecksum() throws IOException {
        Game.Result r = new Game(3, 10).play(5, true);
        Path fileA = tempDir.resolve("a.json");
        Path fileB = tempDir.resolve("b.json");
        Protocol.write(r, fileA);
        Protocol.write(r, fileB);

        assertEquals(Protocol.sha256(fileA), Protocol.sha256(fileB));
    }

    @Test
    void checksumLooksLikeSha256() throws IOException {
```

```
47          Game.Result r = new Game(3, 10).play(5, true);
48          Path file = tempDir.resolve("protocol.json");
49          Protocol.write(r, file);
50
51          String hash = Protocol.sha256(file);
52
53          assertEquals(64, hash.length());
54          assertTrue(hash.matches("[0-9a-f]+"));
55      }
56
57      @Test
58      void differentContentProducesDifferentChecksum() throws IOException {
59          Path file = tempDir.resolve("protocol.json");
60
61          Protocol.write(new Game(3, 10).play(1, true), file);
62          String hashA = Protocol.sha256(file);
63
64          Protocol.write(new Game(3, 10).play(2, true), file);
65          String hashB = Protocol.sha256(file);
66
67          assertTrue(!hashA.equals(hashB));
68      }
69
70      // not a full JSON parser, just a quick sanity check that every
71      // { has a matching } and every [ has a matching ]
72      private static void assertBalancedBracesAndBrackets(String json) {
73          int braces = 0;
74          int brackets = 0;
75          for (char c : json.toCharArray()) {
76              if (c == '{') braces++;
77              if (c == '}') braces--;
78              if (c == '[') brackets++;
79              if (c == ']') brackets--;
80          }
81          assertEquals(0, braces, "unbalanced { }");
82          assertEquals(0, brackets, "unbalanced [ ]");
83      }
84  }
```

L4, L16-17 `@TempDir` asks JUnit to inject a fresh, automatically-cleaned-up temporary directory into the `tempDir` field before each test — so tests never write real files into the project folder.

L19-32 Plays a small game, writes its protocol, reads the file back as UTF-8 text, and checks it starts and ends with the object braces, contains the expected fields, and has balanced braces/brackets (via the helper at L72-83).

L34-43 Writes the exact same `Result` to two different files and checks they hash to the identical SHA-256 — proving the hash depends only on content, not on incidental factors like file path or timing.

L45-55	Checks the checksum is exactly 64 characters and matches the pattern every valid SHA-256 hex digest has.
L57-68	Writes two games played with different seeds to the same file path (overwriting between writes) and checks their checksums differ — confirming the hash is actually sensitive to content.
L70-83	A private helper used by the first test: not a full JSON parser, just a quick structural sanity check that counts opening/closing braces and brackets and confirms they balance to zero — catching the kind of bug where one got dropped.